Table of Contents

1. Executive Summary

Overall Risk Score

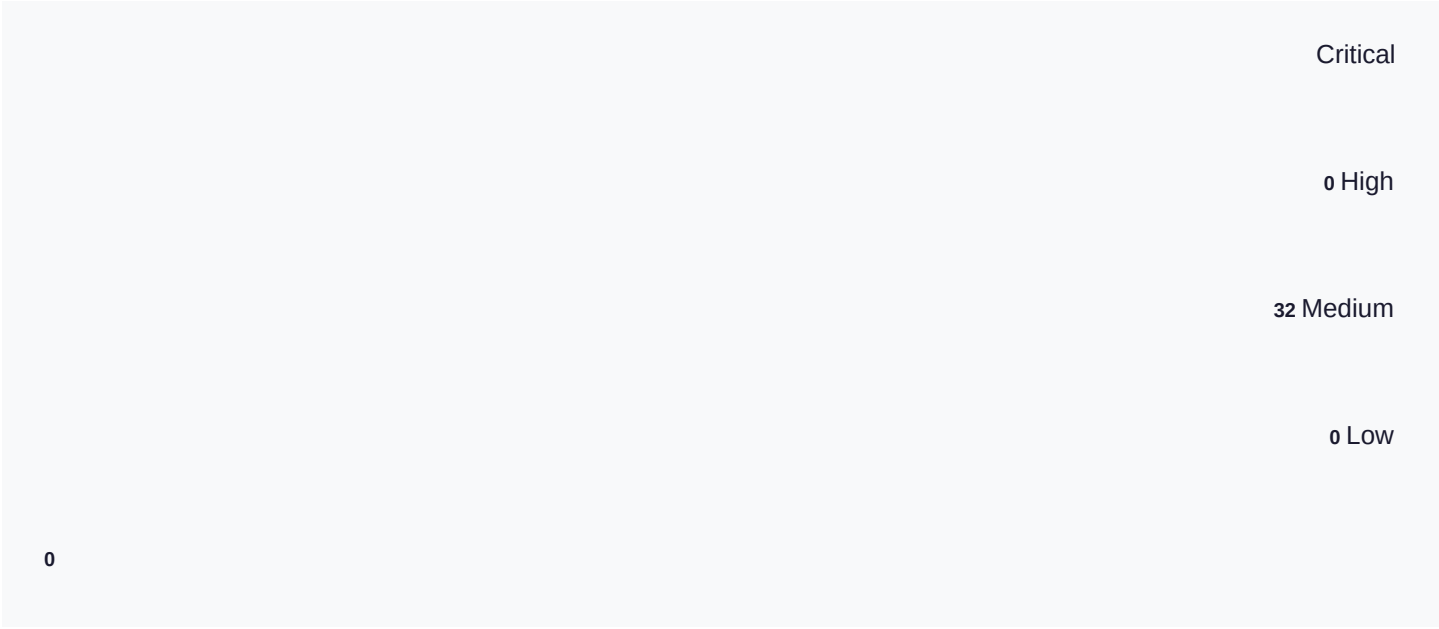
Critical risk - Severe vulnerabilities found. Urgent remediation required.

Property	Value
Target URL	`https://ginandjuice.shop/`

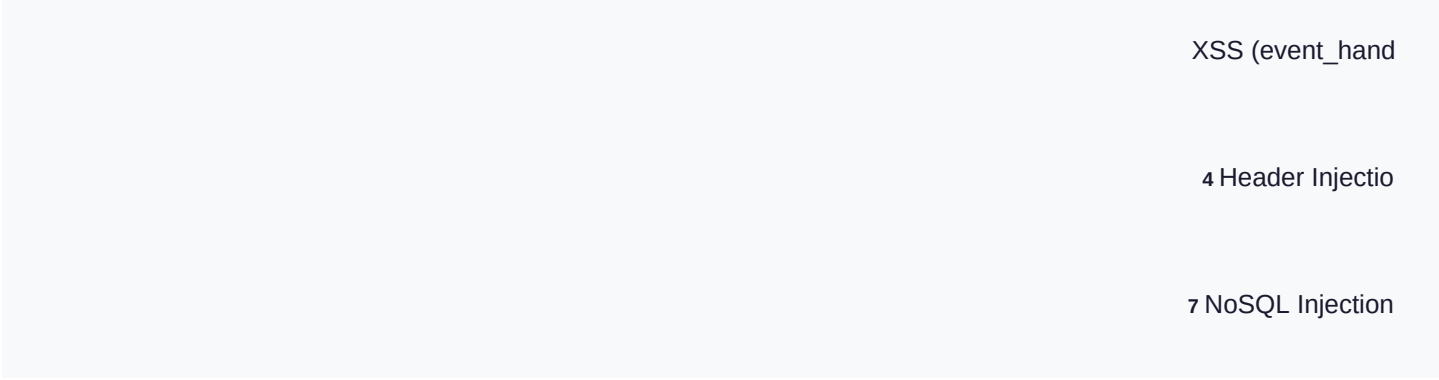
Scan Start	2026-01-30 11:08:55
Scan End	2026-01-30 11:16:49
Status	completed
Total Vulnerabilities	32

2. Vulnerability Overview

Severity Distribution



Vulnerability Types



15 NoSQL Injection

5 SQL Injection (

1

Vulnerabilities Summary Table

#	Type	Severity	URL	Parameter	Confidence
1	XSS (event_handler)	HIGH	`https://ginandjuisearchTermtalog`		80%
2	Header Injection (CRLF)	HIGH	`https://ginandjuisearchTermtalog`		90%
3	NoSQL Injection (operator)	HIGH	`https://ginandjuisearchTermtalog`		85%
4	NoSQL Injection (operator)	HIGH	`https://ginandjuisearchTermtalog`		85%
5	NoSQL Injection (operator)	HIGH	`https://ginandjuisearchTermtalog`		85%
6	NoSQL Injection (where)	HIGH	`https://ginandjuisearchTermtalog`		85%

7	SQL Injection (Boolean-based Blind)	HIGH	`https://ginandjuicategory'catalog`	90%
8	XSS (event_handler)	HIGH	`https://ginandjuicategory'catalog`	95%
9	Header Injection (CRLF)	HIGH	`https://ginandjuicategory'catalog`	90%
10	NoSQL Injection (operator)	HIGH	`https://ginandjuicategory'catalog`	85%
11	NoSQL Injection (operator)	HIGH	`https://ginandjuicategory'catalog`	85%
12	NoSQL Injection (operator)	HIGH	`https://ginandjuicategory'catalog`	85%
13	NoSQL Injection (where)	HIGH	`https://ginandjuicategory'catalog`	85%
14	Header Injection (CRLF)	HIGH	`https://ginandjuisearch%p/blog/`	90%
15	Header Injection (CRLF)	HIGH	`https://ginandjuiback%hop/blog/`	90%

16	NoSQL Injection (operator)	HIGH	`https://ginandjuisearch›p/blog/`	85%
17	NoSQL Injection (operator)	HIGH	`https://ginandjuisearch›p/blog/`	85%
18	NoSQL Injection (operator)	HIGH	`https://ginandjuisearch›p/blog/`	85%
19	NoSQL Injection (where)	HIGH	`https://ginandjuisearch›p/blog/`	85%
20	Header Injection (CRLF)	HIGH	`https://ginandjuiback›hop/blog/`	90%
21	XSS (event_handler)	HIGH	`https://ginandjuisearchTerm‹talog`	80%
22	XSS (event_handler)	HIGH	`https://ginandjuicategory'catalog`	95%
23	Header Injection (CRLF)	HIGH	`https://ginandjuisearchTerm‹talog`	90%
24	Header Injection (CRLF)	HIGH	`https://ginandjuicategory'catalog`	90%

25	NoSQL Injection (operator)	HIGH	`https://ginandjuisearchTermtalog`	85%
26	NoSQL Injection (operator)	HIGH	`https://ginandjuisearchTermtalog`	85%
27	NoSQL Injection (operator)	HIGH	`https://ginandjuisearchTermtalog`	85%
28	NoSQL Injection (where)	HIGH	`https://ginandjuisearchTermtalog`	85%
29	NoSQL Injection (operator)	HIGH	`https://ginandjuicategory'catalog`	85%
30	NoSQL Injection (operator)	HIGH	`https://ginandjuicategory'catalog`	85%
31	NoSQL Injection (operator)	HIGH	`https://ginandjuicategory'catalog`	85%
32	NoSQL Injection (where)	HIGH	`https://ginandjuicategory'catalog`	85%

3. Detailed Findings

1. XSS (event_handler)

HIGH

Affected URL

```
https://ginandjuice.shop/catalog
```

Vulnerable Parameter

```
searchTerm
```

Payload Used

```
">
```

Description

Vulnerability found in parameter 'searchTerm' at https://ginandjuice.shop/catalog

Evidence

Payload '">' triggered vulnerability detection

Confidence Level

80%

How to Fix

- Encode all user input before displaying in HTML (use HTML entity encoding)
- Use Content-Security-Policy (CSP) headers to restrict script sources
- Implement HttpOnly and Secure flags on session cookies
- Use modern frameworks that auto-escape output (React, Vue, Angular)
- Validate and sanitize all input on the server side
- Use textContent instead of innerHTML when setting element content

2. Header Injection (CRLF)

HIGH

Affected URL

```
https://ginandjuice.shop/catalog
```

Vulnerable Parameter

```
searchTerm
```

Payload Used

```
test Set-Cookie: injected=true
```

Description

Vulnerability found in parameter 'searchTerm' at https://ginandjuice.shop/catalog

Evidence

Payload 'test Set-Cookie: injected=true' triggered vulnerability detection

Confidence Level

90%

How to Fix

- Never include raw user input in HTTP headers
- Strip or encode CR (\r) and LF (\n) characters
- Use framework functions that sanitize header values
- Validate input against strict patterns

3. NoSQL Injection (operator)

HIGH

Affected URL

```
https://ginandjuice.shop/catalog
```

Vulnerable Parameter

```
searchTerm
```

Payload Used

```
{"$gt": ""}
```

Description

Vulnerability found in parameter 'searchTerm' at https://ginandjuice.shop/catalog

Evidence

Payload '{"$gt": ""}' triggered vulnerability detection

Confidence Level

85%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

4. NoSQL Injection (operator)

HIGH

Affected URL

```
https://ginandjuice.shop/catalog
```

Vulnerable Parameter

```
searchTerm
```

Payload Used

```
{"$ne": null}
```

Description

Vulnerability found in parameter 'searchTerm' at https://ginandjuice.shop/catalog

Evidence

Payload '{"$ne": null}' triggered vulnerability detection

Confidence Level

85%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

5. NoSQL Injection (operator)

HIGH

Affected URL

```
https://ginandjuice.shop/catalog
```

Vulnerable Parameter

```
searchTerm
```

Payload Used

```
{"$ne": ""}
```

Description

Vulnerability found in parameter 'searchTerm' at https://ginandjuice.shop/catalog

Evidence

Payload '{"$ne": ""}' triggered vulnerability detection

Confidence Level

85%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

6. NoSQL Injection (where)

HIGH

Affected URL

```
https://ginandjuice.shop/catalog
```

Vulnerable Parameter

```
searchTerm
```

Payload Used

```
{"$where": "1==1"}
```

Description

Vulnerability found in parameter 'searchTerm' at https://ginandjuice.shop/catalog

Evidence

Payload '{"$where": "1==1"}' triggered vulnerability detection

Confidence Level

85%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

7. SQL Injection (Boolean-based Blind)

HIGH

Affected URL

```
https://ginandjuice.shop/catalog
```

Vulnerable Parameter

```
category
```

Payload Used

```
' OR '1'='1' --
```

Description

Vulnerability found in parameter 'category' at https://ginandjuice.shop/catalog

Evidence

Payload '' OR '1'='1' --' triggered vulnerability detection

Confidence Level

90%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs

- Implement Web Application Firewall (WAF) as defense-in-depth

8. XSS (event_handler)

HIGH

Affected URL

```
https://ginandjuice.shop/catalog
```

Vulnerable Parameter

```
category
```

Payload Used

```
">
```

Description

Vulnerability found in parameter 'category' at https://ginandjuice.shop/catalog

Evidence

Payload '">' triggered vulnerability detection

Confidence Level

95%

How to Fix

- Encode all user input before displaying in HTML (use HTML entity encoding)
- Use Content-Security-Policy (CSP) headers to restrict script sources
- Implement HttpOnly and Secure flags on session cookies
- Use modern frameworks that auto-escape output (React, Vue, Angular)
- Validate and sanitize all input on the server side
- Use textContent instead of innerHTML when setting element content

9. Header Injection (CRLF)

HIGH

Affected URL

```
https://ginandjuice.shop/catalog
```

Vulnerable Parameter

```
category
```

Payload Used

```
test X-Injected: true
```

Description

Vulnerability found in parameter 'category' at https://ginandjuice.shop/catalog

Evidence

Payload 'test X-Injected: true' triggered vulnerability detection

Confidence Level

90%

How to Fix

- Never include raw user input in HTTP headers
- Strip or encode CR (\r) and LF (\n) characters
- Use framework functions that sanitize header values
- Validate input against strict patterns

10. NoSQL Injection (operator)

HIGH

Affected URL

```
https://ginandjuice.shop/catalog
```

Vulnerable Parameter

```
category
```

Payload Used

```
{"$gt": ""}
```

Description

Vulnerability found in parameter 'category' at https://ginandjuice.shop/catalog

Evidence

Payload '{"$gt": ""}' triggered vulnerability detection

Confidence Level

85%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

11. NoSQL Injection (operator)

HIGH

Affected URL

```
https://ginandjuice.shop/catalog
```

Vulnerable Parameter

```
category
```

Payload Used

```
{"$ne": null}
```

Description

Vulnerability found in parameter 'category' at https://ginandjuice.shop/catalog

Evidence

Payload '{"$ne": null}' triggered vulnerability detection

Confidence Level

85%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

12. NoSQL Injection (operator)

HIGH

Affected URL

```
https://ginandjuice.shop/catalog
```

Vulnerable Parameter

```
category
```

Payload Used

```
{"$ne": ""}
```

Description

Vulnerability found in parameter 'category' at https://ginandjuice.shop/catalog

Evidence

Payload '{"$ne": ""}' triggered vulnerability detection

Confidence Level

85%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

13. NoSQL Injection (where)

HIGH

Affected URL

```
https://ginandjuice.shop/catalog
```

Vulnerable Parameter

```
category
```

Payload Used

```
{"$where": "1==1"}
```

Description

Vulnerability found in parameter 'category' at https://ginandjuice.shop/catalog

Evidence

Payload '{"$where": "1==1"}' triggered vulnerability detection

Confidence Level

85%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

14. Header Injection (CRLF)

HIGH

Affected URL

```
https://ginandjuice.shop/blog/
```

Vulnerable Parameter

```
search
```

Payload Used

```
test Set-Cookie: injected=true
```

Description

Vulnerability found in parameter 'search' at https://ginandjuice.shop/blog/

Evidence

Payload 'test Set-Cookie: injected=true' triggered vulnerability detection

Confidence Level

90%

How to Fix

- Never include raw user input in HTTP headers
- Strip or encode CR (\r) and LF (\n) characters

- Use framework functions that sanitize header values
- Validate input against strict patterns

15. Header Injection (CRLF)

HIGH

Affected URL

```
https://ginandjuice.shop/blog/
```

Vulnerable Parameter

```
back
```

Payload Used

```
test Set-Cookie: injected=true
```

Description

Vulnerability found in parameter 'back' at https://ginandjuice.shop/blog/

Evidence

Payload 'test Set-Cookie: injected=true' triggered vulnerability detection

Confidence Level

90%

How to Fix

- Never include raw user input in HTTP headers
- Strip or encode CR (\r) and LF (\n) characters
- Use framework functions that sanitize header values
- Validate input against strict patterns

16. NoSQL Injection (operator)

HIGH

Affected URL

```
https://ginandjuice.shop/blog/
```

Vulnerable Parameter

```
search
```

Payload Used

```
{"$gt": ""}
```

Description

Vulnerability found in parameter 'search' at https://ginandjuice.shop/blog/

Evidence

Payload '{"$gt": ""}' triggered vulnerability detection

Confidence Level

85%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

17. NoSQL Injection (operator)

HIGH

Affected URL

```
https://ginandjuice.shop/blog/
```

Vulnerable Parameter

```
search
```

Payload Used

```
{"$ne": null}
```

Description

Vulnerability found in parameter 'search' at https://ginandjuice.shop/blog/

Evidence

Payload '{"$ne": null}' triggered vulnerability detection

Confidence Level

85%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

18. NoSQL Injection (operator)

HIGH

Affected URL

```
https://ginandjuice.shop/blog/
```

Vulnerable Parameter

```
search
```

Payload Used

```
{"$ne": ""}
```

Description

Vulnerability found in parameter 'search' at https://ginandjuice.shop/blog/

Evidence

Payload '{"$ne": ""}' triggered vulnerability detection

Confidence Level

85%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

19. NoSQL Injection (where)

HIGH

Affected URL

```
https://ginandjuice.shop/blog/
```

Vulnerable Parameter

```
search
```

Payload Used

```
{"$where": "1==1"}
```

Description

Vulnerability found in parameter 'search' at https://ginandjuice.shop/blog/

Evidence

Payload '{"$where": "1==1"}' triggered vulnerability detection

Confidence Level

85%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

20. Header Injection (CRLF)

HIGH

Affected URL

```
https://ginandjuice.shop/blog/
```

Vulnerable Parameter

```
back
```

Payload Used

```
test Set-Cookie: injected=true
```

Description

Vulnerability found in parameter 'back' at https://ginandjuice.shop/blog/

Evidence

Payload 'test Set-Cookie: injected=true' triggered vulnerability detection

Confidence Level

90%

How to Fix

- Never include raw user input in HTTP headers
- Strip or encode CR (\r) and LF (\n) characters
- Use framework functions that sanitize header values
- Validate input against strict patterns

21. XSS (event_handler)

HIGH

Affected URL

```
https://ginandjuice.shop/catalog
```

Vulnerable Parameter

```
searchTerm
```

Payload Used

```
">
```

Description

Vulnerability found in parameter 'searchTerm' at https://ginandjuice.shop/catalog

Evidence

Payload '">' triggered vulnerability detection

Confidence Level

80%

How to Fix

- Encode all user input before displaying in HTML (use HTML entity encoding)
- Use Content-Security-Policy (CSP) headers to restrict script sources
- Implement HttpOnly and Secure flags on session cookies
- Use modern frameworks that auto-escape output (React, Vue, Angular)

- Validate and sanitize all input on the server side
- Use textContent instead of innerHTML when setting element content

22. XSS (event_handler)

HIGH

Affected URL

```
https://ginandjuice.shop/catalog
```

Vulnerable Parameter

```
category
```

Payload Used

```
">
```

Description

Vulnerability found in parameter 'category' at https://ginandjuice.shop/catalog

Evidence

Payload '">' triggered vulnerability detection

Confidence Level

95%

How to Fix

- Encode all user input before displaying in HTML (use HTML entity encoding)
- Use Content-Security-Policy (CSP) headers to restrict script sources
- Implement HttpOnly and Secure flags on session cookies
- Use modern frameworks that auto-escape output (React, Vue, Angular)
- Validate and sanitize all input on the server side
- Use textContent instead of innerHTML when setting element content

23. Header Injection (CRLF)

HIGH

Affected URL

```
https://ginandjuice.shop/catalog
```

Vulnerable Parameter

```
searchTerm
```

Payload Used

```
test Set-Cookie: injected=true
```

Description

Vulnerability found in parameter 'searchTerm' at https://ginandjuice.shop/catalog

Evidence

Payload 'test Set-Cookie: injected=true' triggered vulnerability detection

Confidence Level

90%

How to Fix

- Never include raw user input in HTTP headers
- Strip or encode CR (\r) and LF (\n) characters
- Use framework functions that sanitize header values
- Validate input against strict patterns

24. Header Injection (CRLF)

HIGH

Affected URL

```
https://ginandjuice.shop/catalog
```

Vulnerable Parameter

```
category
```

Payload Used

```
test X-Injected: true
```

Description

Vulnerability found in parameter 'category' at https://ginandjuice.shop/catalog

Evidence

Payload 'test X-Injected: true' triggered vulnerability detection

Confidence Level

90%

How to Fix

- Never include raw user input in HTTP headers
- Strip or encode CR (\r) and LF (\n) characters
- Use framework functions that sanitize header values
- Validate input against strict patterns

25. NoSQL Injection (operator)

HIGH

Affected URL

```
https://ginandjuice.shop/catalog
```

Vulnerable Parameter

```
searchTerm
```

Payload Used

```
{"$gt": ""}
```

Description

Vulnerability found in parameter 'searchTerm' at https://ginandjuice.shop/catalog

Evidence

Payload '{"$gt": ""}' triggered vulnerability detection

Confidence Level

85%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

26. NoSQL Injection (operator)

HIGH

Affected URL

```
https://ginandjuice.shop/catalog
```

Vulnerable Parameter

```
searchTerm
```

Payload Used

```
{"$ne": null}
```

Description

Vulnerability found in parameter 'searchTerm' at https://ginandjuice.shop/catalog

Evidence

Payload '{"$ne": null}' triggered vulnerability detection

Confidence Level

85%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

27. NoSQL Injection (operator)

HIGH

Affected URL

```
https://ginandjuice.shop/catalog
```

Vulnerable Parameter

```
searchTerm
```

Payload Used

```
{"$ne": ""}
```

Description

Vulnerability found in parameter 'searchTerm' at https://ginandjuice.shop/catalog

Evidence

Payload '{"$ne": ""}' triggered vulnerability detection

Confidence Level

85%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

28. NoSQL Injection (where)

HIGH

Affected URL

```
https://ginandjuice.shop/catalog
```

Vulnerable Parameter

```
searchTerm
```

Payload Used

```
{"$where": "1==1"}
```

Description

Vulnerability found in parameter 'searchTerm' at https://ginandjuice.shop/catalog

Evidence

Payload '{"$where": "1==1"}' triggered vulnerability detection

Confidence Level

85%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically

- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

29. NoSQL Injection (operator)

HIGH

Affected URL

```
https://ginandjuice.shop/catalog
```

Vulnerable Parameter

```
category
```

Payload Used

```
{"$gt": ""}
```

Description

Vulnerability found in parameter 'category' at https://ginandjuice.shop/catalog

Evidence

Payload '{"$gt": ""}' triggered vulnerability detection

Confidence Level

85%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

30. NoSQL Injection (operator)

HIGH

Affected URL

```
https://ginandjuice.shop/catalog
```

Vulnerable Parameter

```
category
```

Payload Used

```
{"$ne": null}
```

Description

Vulnerability found in parameter 'category' at https://ginandjuice.shop/catalog

Evidence

Payload '{"$ne": null}' triggered vulnerability detection

Confidence Level

85%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

31. NoSQL Injection (operator)

HIGH

Affected URL

```
https://ginandjuice.shop/catalog
```

Vulnerable Parameter

```
category
```

Payload Used

```
{"$ne": ""}
```

Description

Vulnerability found in parameter 'category' at https://ginandjuice.shop/catalog

Evidence

Payload '{"$ne": ""}' triggered vulnerability detection

Confidence Level

85%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

32. NoSQL Injection (where)

HIGH

Affected URL

```
https://ginandjuice.shop/catalog
```

Vulnerable Parameter

```
category
```

Payload Used

```
{"$where": "1==1"}
```

Description

Vulnerability found in parameter 'category' at https://ginandjuice.shop/catalog

Evidence

Payload '{"$where": "1==1"}' triggered vulnerability detection

Confidence Level

85%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

4. Remediation Guide

This section provides detailed guidance on how to fix the vulnerabilities found during the scan. Each vulnerability type includes an explanation of how the attack works and code examples for remediation.

XSS (event_handler)

4 Finding(s)

What is XSS (event_handler)?

Cross-Site Scripting (XSS) occurs when an application includes untrusted data in a web page without proper validation or escaping. This allows attackers to execute malicious scripts in victims' browsers.

How the Attack Works

1. Attacker identifies an input field that reflects user data in the page 2. Attacker crafts a malicious payload containing JavaScript code 3. When a victim views the page, the script executes in their browser 4. The script can steal cookies, session tokens, or perform actions as the user

Potential Impact

- Session hijacking and account takeover
- Theft of sensitive data (cookies, tokens)
- Defacement of web pages
- Phishing attacks using trusted domain
- Malware distribution

Remediation Steps

- Encode all user input before displaying in HTML (use HTML entity encoding)
- Use Content-Security-Policy (CSP) headers to restrict script sources
- Implement HttpOnly and Secure flags on session cookies

- Use modern frameworks that auto-escape output (React, Vue, Angular)
- Validate and sanitize all input on the server side
- Use textContent instead of innerHTML when setting element content

Code Examples

VULNERABLE CODE:

SECURE CODE:

```
// Server-side (Python/Flask example): from markupsafe import escape @app.route('/display') def display(): user_input = request.args.get('input', '') safe_input = escape(user_input) return render_template('page.html', content=safe_input)
```

References

- https://owasp.org/www-community/attacks/xss/
- https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

CWE: CWE-79 **OWASP:** A03:2021 - Injection

Header Injection (CRLF)

7 Finding(s)

What is Header Injection (CRLF)?

Header Injection occurs when user input is included in HTTP response headers without proper sanitization. By injecting CRLF (\r\n) sequences, attackers can add arbitrary headers or split the response.

How the Attack Works

1. Application includes user input in response headers 2. Attacker provides input with CRLF characters 3. New headers are injected into the response 4. Can lead to XSS, cache poisoning, or session fixation

Potential Impact

- Cross-site scripting via injected content
- Cache poisoning
- Session fixation attacks
- HTTP response splitting

Remediation Steps

- Never include raw user input in HTTP headers
- Strip or encode CR (\r) and LF (\n) characters
- Use framework functions that sanitize header values
- Validate input against strict patterns

Code Examples

VULNERABLE CODE:

```
# VULNERABLE CODE (Python/Flask) @app.route('/redirect') def redirect_user(): location = request.args.get('url') response = make_response() response.headers['Location'] = location return response # Attacker: ?url=http://example.com%0d%0aSet-Cookie:%20session=evil
```

SECURE CODE:

```
# SECURE CODE (Python/Flask) import re @app.route('/redirect') def redirect_user(): location = request.args.get('url', '/') # Remove any CRLF characters safe_location = re.sub(r'[\r\n]', '', location) # Or validate URL format strictly if not re.match(r'^https?://[a-zA-Z0-9.-]+/', safe_location): safe_location = '/' return redirect(safe_location)
```

References

- https://owasp.org/www-community/attacks/HTTP_Response_Splitting

CWE: CWE-113 **OWASP:** A03:2021 - Injection

NoSQL Injection (operator)

15 Finding(s)

What is NoSQL Injection (operator)?

SQL Injection occurs when untrusted data is sent to an interpreter as part of a command or query. The attacker's hostile data can trick the interpreter into executing unintended commands or accessing data without authorization.

How the Attack Works

1. Application builds SQL queries using string concatenation with user input 2. Attacker provides malicious input containing SQL syntax 3. The database executes the modified query 4. Attacker can read, modify, or delete data; bypass authentication

Potential Impact

- Complete database compromise
- Data theft (credentials, personal info, financial data)
- Data modification or deletion
- Authentication bypass
- Remote code execution (in some cases)
- Full server compromise

Remediation Steps

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

Code Examples

VULNERABLE CODE:

```
# VULNERABLE CODE (Python) query = "SELECT * FROM users WHERE username = '" + username + "'"
cursor.execute(query) # VULNERABLE CODE (PHP) $query = "SELECT * FROM users WHERE id = " .
$_GET['id']; $result = mysqli_query($conn, $query);
```

SECURE CODE:

```
# SECURE CODE (Python with parameterized query) query = "SELECT * FROM users WHERE username = ?"
cursor.execute(query, (username,)) # SECURE CODE (Python with SQLAlchemy ORM) user =
session.query(User).filter(User.username == username).first() # SECURE CODE (PHP with PDO) $stmt =
$pdo->prepare("SELECT * FROM users WHERE id = ?"); $stmt->execute([$_GET['id']]);
```

References

- https://owasp.org/www-community/attacks/SQL_Injection
- https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

CWE: CWE-89 **OWASP:** A03:2021 - Injection

NoSQL Injection (where)

5 Finding(s)

What is NoSQL Injection (where)?

SQL Injection occurs when untrusted data is sent to an interpreter as part of a command or query. The attacker's hostile data can trick the interpreter into executing unintended commands or accessing data without authorization.

How the Attack Works

1. Application builds SQL queries using string concatenation with user input 2. Attacker provides malicious input containing SQL syntax 3. The database executes the modified query 4. Attacker can read, modify, or delete data; bypass authentication

Potential Impact

- Complete database compromise
- Data theft (credentials, personal info, financial data)
- Data modification or deletion
- Authentication bypass
- Remote code execution (in some cases)
- Full server compromise

Remediation Steps

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

Code Examples

VULNERABLE CODE:

```
# VULNERABLE CODE (Python) query = "SELECT * FROM users WHERE username = '" + username + "'"
cursor.execute(query) # VULNERABLE CODE (PHP) $query = "SELECT * FROM users WHERE id = " .
$_GET['id']; $result = mysqli_query($conn, $query);
```

SECURE CODE:

```
# SECURE CODE (Python with parameterized query) query = "SELECT * FROM users WHERE username = ?"
cursor.execute(query, (username,)) # SECURE CODE (Python with SQLAlchemy ORM) user =
session.query(User).filter(User.username == username).first() # SECURE CODE (PHP with PDO) $stmt =
$pdo->prepare("SELECT * FROM users WHERE id = ?"); $stmt->execute([$_GET['id']]);
```

References

- https://owasp.org/www-community/attacks/SQL_Injection
- https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

CWE: CWE-89 **OWASP:** A03:2021 - Injection

SQL Injection (Boolean-based Blind)

1 Finding(s)

What is SQL Injection (Boolean-based Blind)?

SQL Injection occurs when untrusted data is sent to an interpreter as part of a command or query. The attacker's hostile data can trick the interpreter into executing unintended commands or accessing data without authorization.

How the Attack Works

1. Application builds SQL queries using string concatenation with user input 2. Attacker provides malicious input containing SQL syntax 3. The database executes the modified query 4. Attacker can read, modify, or delete data; bypass authentication

Potential Impact

- Complete database compromise
- Data theft (credentials, personal info, financial data)
- Data modification or deletion
- Authentication bypass

- Remote code execution (in some cases)
- Full server compromise

Remediation Steps

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

Code Examples

VULNERABLE CODE:

```
# VULNERABLE CODE (Python) query = "SELECT * FROM users WHERE username = '" + username + "'" cursor.execute(query) # VULNERABLE CODE (PHP) $query = "SELECT * FROM users WHERE id = " . $_GET['id']; $result = mysqli_query($conn, $query);
```

SECURE CODE:

```
# SECURE CODE (Python with parameterized query) query = "SELECT * FROM users WHERE username = ?" cursor.execute(query, (username,)) # SECURE CODE (Python with SQLAlchemy ORM) user = session.query(User).filter(User.username == username).first() # SECURE CODE (PHP with PDO) $stmt = $pdo->prepare("SELECT * FROM users WHERE id = ?"); $stmt->execute([$_GET['id']]);
```

References

- https://owasp.org/www-community/attacks/SQL_Injection
- https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

CWE: CWE-89 **OWASP:** A03:2021 - Injection

5. Methodology

This automated scan was performed using the Web Vulnerability Scanner, which employs the following techniques to discover and test for vulnerabilities:

Scanning Phases

Phase	Description
1. Discovery	Crawls the target website to discover pages, forms, API endpoints, and parameters using browser automation (Playwright).
2. Analysis	Analyzes discovered elements to identify potential injection points and build a map of testable targets.

3. Testing	Tests each target with specialized payloads for various vulnerability types (XSS, SQLi, etc.).
4. Validation	Confirms findings using multi-stage validation to reduce false positives. Assigns confidence scores.

Vulnerability Types Tested

Cross-Site Scripting (XSS) SQL Injection NoSQL Injection Path Traversal Open Redirect SSTI Command Injection SSRF XXE Header Injection

Disclaimer: This automated scan may not detect all vulnerabilities. Manual penetration testing by qualified security professionals is recommended for comprehensive security assessment. Always obtain proper authorization before scanning.